

Nova OS – Band 1: Architektur

Version: 0.1

Status: Entwurf

Codename: Nova Architecture Core

1. Zweck

Band 1 definiert die technische Gesamtarchitektur von Nova OS.

Dieses Dokument ist die oberste Referenz für:

- Projektstruktur
- Systemarchitektur
- Modulaufteilung
- Kernelgrenzen
- Bootprozess
- Sicherheitsmodell
- Grafikarchitektur
- Buildsystem
- Entwicklungsregeln
- Dokumentationsstandard

Keine größere Komponente wird implementiert, bevor ihre Architektur mit diesem Dokument vereinbar ist.

2. Leitbild

Nova OS ist ein modernes, modulares, lokales und intelligentes Betriebssystem.

Es verbindet:

- hohe Performance
- elegante Benutzeroberfläche
- lokale KI
- Datenschutz
- adaptive Hardware-Anpassung
- lange Wartbarkeit
- klare Architektur

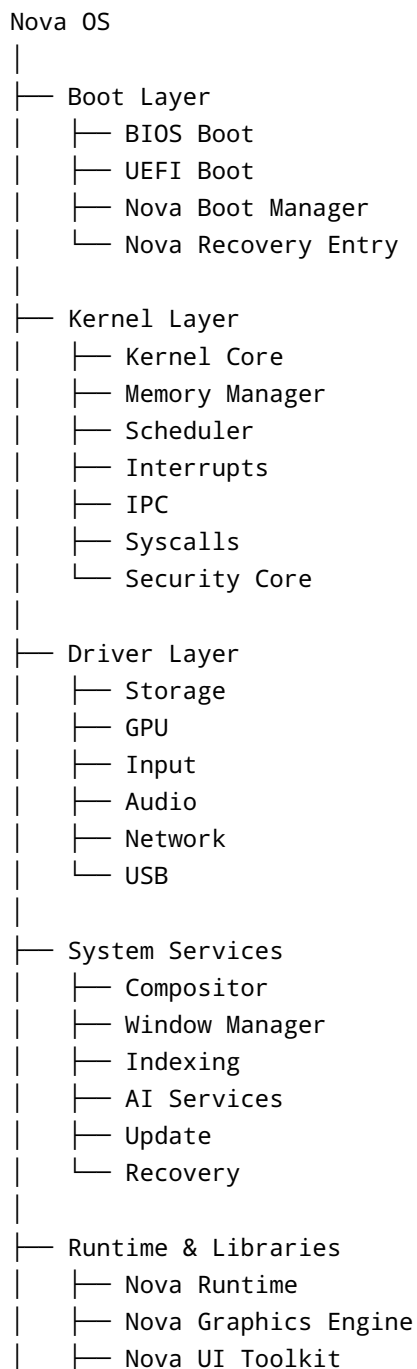
Nova OS soll nicht bestehende Betriebssysteme kopieren, sondern eine eigene Plattform bilden.

3. Grundprinzipien

1. Architektur vor Code
2. Modularität vor Monolith

3. Sicherheit vor Komfort
 4. Datenschutz vor Cloud
 5. Lokale Verarbeitung zuerst
 6. Keine versteckten Abhängigkeiten
 7. Keine magischen Zahlen
 8. Keine festen Bildschirmauflösungen
 9. Dokumentation ist Teil des Produkts
 10. Jede Komponente muss austauschbar oder klar begründet fest gekoppelt sein
-

4. Hauptarchitektur



```
|   └─ Nova API
|
└─ User Layer
    ├── Desktop
    ├── Apps
    ├── Settings
    ├── Explorer
    └─ Assistant
```

5. Repository-Struktur

```
NovaOS/
├─ CMakeLists.txt
├─ README.md
├─ LICENSE
├─ CHANGELOG.md
├─ CONTRIBUTING.md
├─ .gitignore
├─ .clang-format
├─ .editorconfig
├─
├─ docs/
│   ├── band_01_architektur/
│   ├── band_02_bootloader/
│   ├── band_03_kernel/
│   ├── band_04_memory/
│   ├── band_05_graphics/
│   ├── band_06_driver/
│   ├── band_07_filesystem/
│   ├── band_08_network/
│   ├── band_09_recovery/
│   ├── band_10_security/
│   ├── band_11_desktop/
│   ├── band_12_ai/
│   ├── band_13_sdk/
│   └─ band_14_installer/
├─
├─ boot/
├─ kernel/
├─ runtime/
├─ libraries/
├─ drivers/
├─ services/
├─ system/
├─ recovery/
├─ installer/
├─ sdk/
```

```
|─ applications/  
|─ assets/  
|─ themes/  
|─ tools/  
|─ tests/  
└─ third_party/
```

6. Kernel-Grenze

Der Kernel enthält nur Komponenten, die zwingend privilegiert laufen müssen.

In den Kernel gehören:

- CPU-Initialisierung
- Speicherverwaltung
- Paging
- Scheduler
- Prozesse
- Threads
- Interrupts
- Syscalls
- IPC
- Kernelobjekte
- Sicherheitskern
- Treiberbasis

Nicht in den Kernel gehören:

- Desktop
- App Store
- KI-Assistent
- Dateimanager
- grafische Einstellungen
- Widgets
- normale Anwendungen
- vollständige Recovery-Oberfläche

7. Bootarchitektur

```
BIOS / UEFI  
  ↓  
Nova Boot Manager  
  ↓  
2-Sekunden-Fenster für F5 / F8 / ESC  
  ↓  
Option A: Kernel starten
```

Option B: Recovery starten
Option C: Diagnose starten
Option D: Bootmenü öffnen

Der Bootloader bleibt klein und robust.

Komplexe Funktionen wie Backup, Wiederherstellung, Selbstheilung, Formatierung und Verschlüsselung laufen nicht direkt im Bootloader, sondern in der **Nova Recovery Environment**.

8. Sicherheitsmodell

Nova OS folgt dem Prinzip:

Standardmäßig sicher.
Optional offen.
Niemals heimlich.

Pflichtregeln:

- Keine Cloud-Nutzung ohne Zustimmung.
- Keine Telemetrie ohne Zustimmung.
- Keine versteckten Hintergrunddienste.
- Systemkomponenten werden signiert.
- Boot-Passwort ist optional.
- TPM-Unterstützung ist optional.
- Vollverschlüsselung ist optional.
- Recovery-Menü kann geschützt werden.

9. Grafikarchitektur

Nova OS verwendet die **Nova Graphics Engine**, kurz **NGE**.

Die NGE besteht aus:

Nova Graphics Engine

- └─ Framebuffer Manager
- └─ Buffer Manager
- └─ Adaptive View System
- └─ Primitive Renderer
- └─ Text Renderer
- └─ Image Renderer
- └─ Effects Engine
- └─ Glass Engine
- └─ Layer System

└─ Compositor
└─ GPU Backend

Regeln:

- Keine festen Auflösungen.
- Keine festen UI-Koordinaten ohne View-System.
- Rendering erfolgt über Buffer.
- Direkter Framebuffer-Zugriff nur im Framebuffer-Modul.
- UI arbeitet später mit skalierbaren Einheiten.
- Querformat, Hochformat und Ultra-Wide müssen unterstützt werden.

10. Adaptive View Engine

Die Adaptive View Engine erkennt automatisch:

- Bildschirmbreite
- Bildschirmhöhe
- Seitenverhältnis
- Hochformat
- Querformat
- Ultra-Wide
- kleine Displays
- hohe DPI
- sichere Ränder

Referenzauflösung:

1920 × 1080

Alle UI-Elemente werden relativ dazu skaliert.

11. Modul-Lebenszyklus

Jedes Modul besitzt einen Status:

ENTWURF
SPEZIFIZIERT
IN ENTWICKLUNG
TESTBAR
STABIL
VERALTET
ENTFERNT

Ein Modul darf erst implementiert werden, wenn mindestens diese Punkte definiert sind:

- Zweck
- Verantwortlichkeit
- Abhängigkeiten
- öffentliche API
- Fehlercodes
- Tests
- Sicherheitsauswirkungen

12. Namenskonventionen

Dateien verwenden das Präfix `nova_`.

Beispiele:

```
nova_kernel.c
nova_kernel.h
nova_framebuffer.c
nova_framebuffer.h
nova_logger.c
nova_logger.h
```

Funktionen verwenden dieses Schema:

```
nova_modul_aktion()
```

Beispiele:

```
nova_logger_info()
nova_framebuffer_init()
nova_scheduler_start()
nova_memory_alloc_page()
```

13. Fehler- und Loggingstandard

Log-Level:

```
DEBUG
INFO
WARNUNG
FEHLER
```

KRITISCH
ERFOLG

Format:

```
[NOVA][INFO] Kernel wurde gestartet.  
[NOVA][WARNUNG] Kein Mausgerät gefunden.  
[NOVA][FEHLER] Framebuffer konnte nicht initialisiert werden.
```

Alle sichtbaren Meldungen sind auf Deutsch.

14. Buildsystem

Standard-Buildsystem:

CMake

Geplante Build-Ziele:

```
nova_bootloader_uefi  
nova_bootloader_bios  
nova_kernel  
nova_recovery  
nova_image  
nova_tests
```

Compiler-Warnungen werden als Fehler behandelt.

Standardoptionen:

```
-Wall  
-Wextra  
-Werror  
-ffreestanding  
-fno-builtin  
-fno-stack-protector
```

15. Dokumentationsstandard

Jedes große Modul erhält:


```
README.md
ARCHITECTURE.md
API.md
TESTS.md
CHANGELOG.md
```

Jede wichtige Architekturentscheidung erhält eine NDA-Datei.

Beispiel:

```
docs/decisions/NDA-0001-triple-buffering.md
docs/decisions/NDA-0002-adaptive-view-engine.md
```

16. Nova Design Authority

Architekturentscheidungen werden dauerhaft dokumentiert.

Format:

```
NDA-0001

Titel:
Triple Buffering als Standard

Status:
Angenommen

Entscheidung:
Nova OS verwendet Triple Buffering als Standardmodell.

Begründung:
- weniger Flackern
- bessere Animationen
- Vorbereitung für Compositor
- Vorbereitung für GPU-Rendering
```

17. Request for Change

Größere Änderungen erfolgen über RFCs.

Format:

RFC-0001

Titel:
Glass Engine 2.0

Status:
Entwurf

Beschreibung:
Neue Rendering-Pipeline für Glas- und Tiefeneffekte.

Betroffene Module:
- NGE
- Compositor
- Theme Engine
- Window Manager

18. Entwicklungsreihenfolge

Die erste offizielle Entwicklungsreihenfolge lautet:

1. Repository-Struktur
2. Buildsystem
3. UEFI Bootloader
4. BIOS Bootloader-Grundlage
5. Boot Context
6. Kernel Entry
7. Logger
8. Framebuffer Manager
9. Adaptive View Engine
10. Buffer Manager
11. Primitive Renderer
12. Kernel-Konsole
13. CPU-Erkennung
14. GDT
15. IDT
16. Paging
17. Heap
18. Scheduler

19. Aktueller Status

Bisher definiert:

- Nova OS Vision
- Architekturprinzipien

- Repository-Struktur
 - Bootloader-Grundkonzept
 - Bootmenü-Konzept
 - Kernel Entry
 - Boot Context
 - Framebuffer Manager
 - Adaptive View Engine
 - Nova Graphics Engine
 - Architekturhandbuch
-

20. Nächster Schritt

Als nächstes wird Band 1 erweitert um:

1. vollständige Root-Dateien
2. vollständige Ordnerstruktur
3. erste CMake-Struktur
4. Coding Standard
5. Modul-Spezifikationsvorlage
6. NDA-Vorlagen
7. RFC-Vorlagen